Software Design Analysis Report

A. Project Details

Project title: AI-Enhanced Learning Management System

Team / roll numbers: Vanshika Garg(2500290109027), Vinayak Gupta(2500290109028), Satya Prakash Kumar(2500290109026), Rajneesh Kumar(2500290109022), Sarita Verma(2500290109025)

Project objective (2–3 lines): To provide a web-based platform for course delivery, learning materials, assessments, grading, communication and progress tracking. AI learning assistance and plagiarism-checking integrations are intended to support personalized learning and academic integrity.

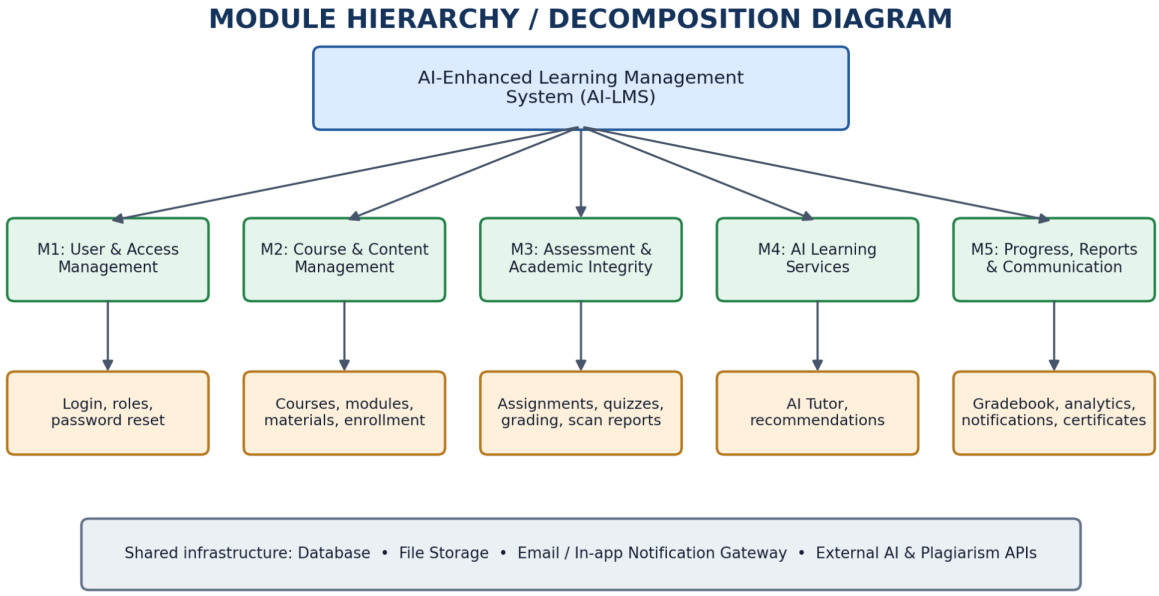
Technology stack: React + Node.js + Express + MongoDB and Python/FastAPI for AI functionality.

B. Module Identification and Modularization

Total number of modules: 5 major logical modules.

Module ID / name	Main responsibility	Inputs / outputs	Depends on
M1 — User & Access Management	Registration, login/logout, password reset, role-based access and administrator user management.	Inputs: credentials, profile/role data. Outputs: authenticated identity, permissions, account status.	Database; email/identity service where configured.
M2 — Course & Content Management	Course creation/publication, modules/lessons, materials, enrollment and completion tracking.	Inputs: course metadata, files/links, enrollment requests. Outputs: course catalogue, lesson content, enrollment and completion status.	M1; database; file storage.
M3 — Assessment & Academic Integrity	Assignments, submissions, quizzes, objective grading, similarity scans, reports and instructor grading.	Inputs: assessment settings, student answers/files. Outputs: scores, feedback, scan status and similarity report.	M1, M2; database/file storage; plagiarism API.
M4 — AI Learning Services	Course-context AI Tutor, AI response feedback, performance-based recommendations and AI enable/disable controls.	Inputs: student query, relevant course context, performance signals. Outputs: AI response and recommendations.	M1, M2, M3; external AI/LLM provider.
M5 — Progress, Reports & Communication	Gradebook, student progress, admin reports, announcements, notifications and course certificates.	Inputs: grades, completion events, announcements. Outputs: reports, notices, certificates and exports.	M1–M3; database; email/in-app notification gateway.

Module hierarchy / decomposition diagram:



C. Design Strategy

Selected strategy: ✅ Top-Down ☐ Bottom-Up ✅ Hybrid

Justification:

- The SRS defines the system scope, actors and functional requirements before implementation details, making top-down decomposition suitable.
- The system can first be divided into access, course, assessment, AI and reporting responsibilities, then each module can be refined into smaller components.
- A hybrid approach is appropriate because external services such as the AI/LLM provider, plagiarism detector and notification gateway may be integrated as reusable components.
- The design keeps provider-specific logic behind service interfaces so that a provider can be changed without redesigning the whole LMS.
- This is a proposed design process; the supplied SRS does not document a completed implementation sequence.

Evidence – Decomposition/build-up sketch:

D. Design Approach

Selected approach: ☐ Function-Oriented ✅ Object-Oriented ✅ Mixed

Justification:

- The main domain concepts are naturally represented as objects, including User, Student, Teacher, Administrator, Course, Module, LearningMaterial, Assignment, Submission, Quiz, Result, Progress and Notification.
- Role-based behavior can be organized around user roles while shared authentication and authorization services are reused.
- Core services such as course management, grading, plagiarism scanning and AI assistance can expose well-defined functions or APIs.
- A mixed approach combines object-oriented domain models with service-oriented functions for external integrations and workflow operations.
- The SRS identifies functional requirements and entities, but does not mandate a specific programming language or framework.

Evidence – High-level service architecture

Layer / component	Role
Presentation layer	Web pages for students, instructors and administrators.
Application/API layer	Authentication, authorization, validation and orchestration of use cases.
Domain modules	Course/content, assessment/plagiarism, AI learning, reporting/communication.
Data and integration layer	Database, file storage, AI/LLM API, plagiarism API and notification gateway.

E. Coupling Analysis

The goal is to keep dependencies explicit and minimize shared internal state. The following are proposed module interactions.

Module pair	Interaction / shared data	Coupling type / level	How to reduce dependency
M1 User & Access ↔ M2 Course & Content	Authenticated user ID, role/permission result, enrollment identity.	Data coupling; low to moderate.	Pass user ID and authorization result through API/service contracts; avoid direct access to M1 internals.
M3 Assessment ↔ Plagiarism service adapter	Submission ID, extracted text/document, scan request, status and similarity report.	Data coupling through external-service interface; moderate.	Use a stable adapter interface, normalized report schema, timeout handling and retry policy.
M4 AI Learning ↔ AI provider adapter	Student question, permitted course context, request ID and generated response.	Data coupling through adapter; moderate.	Keep provider SDK and prompt construction inside adapter; send minimum required data and validate output.
M5 Notifications ↔ M1/M2/M3	Recipient ID and event metadata such as grade posted, announcement or scan completed.	Event/message coupling; low.	Publish domain events or use a notification service contract; avoid direct database-table access.
M5 Reporting ↔ M3 Assessment	Course/assignment IDs, grades, completion status and aggregate data.	Data coupling; moderate.	Use authorized query/API contracts and aggregate only required fields.

F. Cohesion Analysis

Each module should have one clear purpose. The ratings below are design assessments based on the proposed decomposition, not measurements from a running system.

Module	Main responsibility	Cohesion type / level	Improvement, if needed
M1 — User & Access Management	Manage account lifecycle, authentication and role permissions.	Functional cohesion — high.	Keep authentication separate from course-specific business rules; use a central authorization interface.
M2 — Course & Content Management	Manage courses, learning structure, materials and enrollment.	Functional cohesion — high.	Keep file-storage implementation behind a storage interface and separate enrollment rules from content upload.
M3 — Assessment & Academic Integrity	Manage assessment setup, submission, grading and plagiarism workflow.	Sequential/functional cohesion — high if workflow boundaries remain clear.	Keep plagiarism-provider communication in an adapter; separate quiz scoring from document similarity analysis internally if complexity grows.
M4 — AI Learning Services	Provide course-context AI assistance and learning recommendations.	Functional cohesion — high.	Separate AI Tutor and recommendation components behind a common AI integration boundary; label AI output and handle provider failures.
M5 — Progress, Reports & Communication	Provide gradebook/progress views, administrative reports, notifications and certificates.	Communicational cohesion — moderate to high because these features use shared progress/events.	If the module grows, split notifications from analytics/reporting while preserving a shared event interface.

G. Module Interaction and Design Quality

Sequence Diagram:

